Linear Regression

Linear Regression

Consider a training dataset (X_{tr}, y_{tr})

where X_{tr} is an *n-by-a* array and y is an *n-by-1* array

In linear regression, we want to find a scalar *b* and an *a-by-1* array *W* such that

$X_{tr}W + b$ is as close as possible to y_{tr}

hoping that, given a test set (X_{ts}, y_{ts})

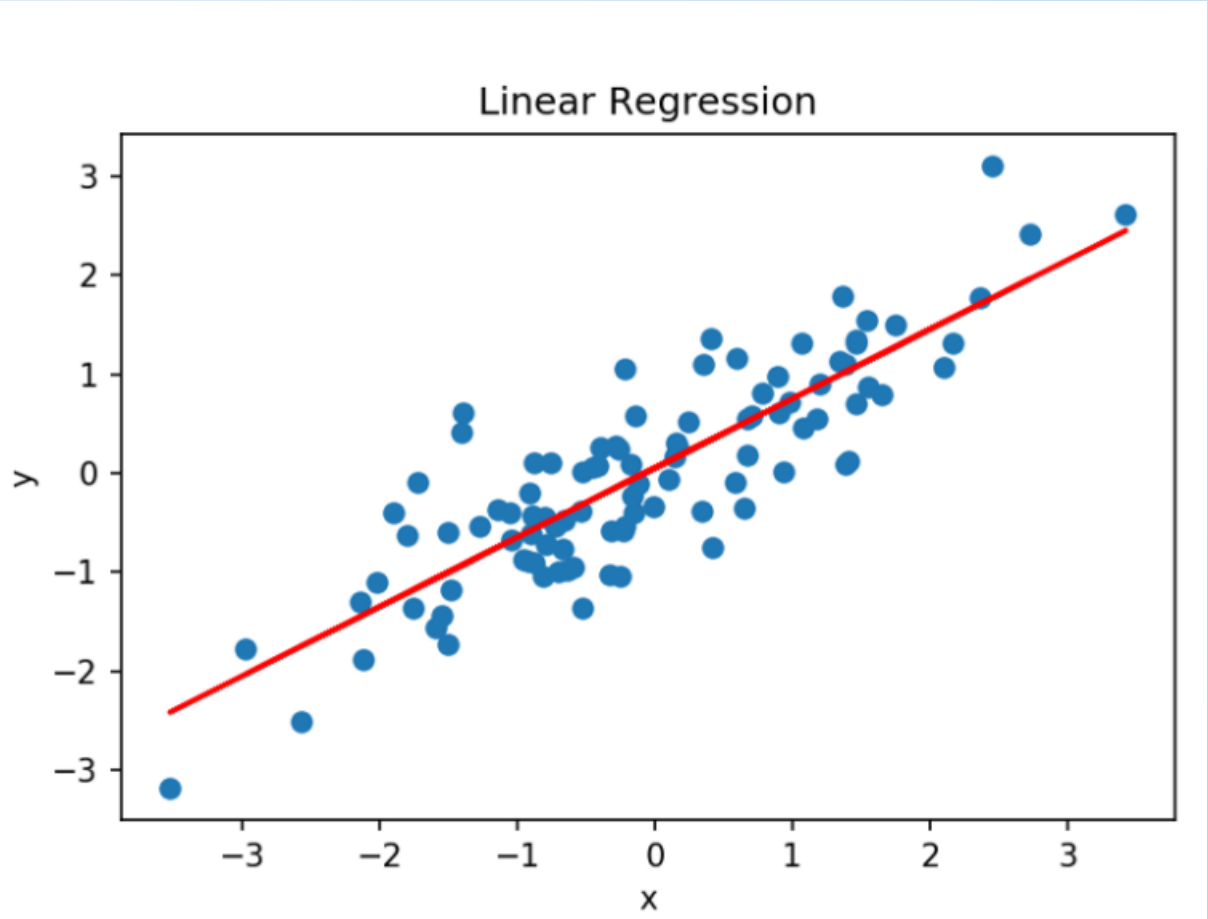
$X_{ts}W + b$ will be close to y_{ts}

Formally, we want to find the values of W and b that minimize

$(X_{tr}W + b - y_{tr})^2$

Linear Regression

1-dimensional example



Linear Regression

We can set $b = \mu(y_{tr})$

That way, if w = [0,0,0,...,0], we will predict the mean value of y in the training set, for all test examples, which as we have seen before, is the best possible constant baseline

$$X_{tr}W + b = y_{tr}$$

$$X_{tr}W = y_{tr} - \mu(y_{tr})$$

If X_{tr} is an invertible matrix (it never is) then

$$(X_{tr})^{-1} X_{tr}W = (X_{tr})^{-1} (y_{tr} - \mu(y_{tr}))$$

and

$$W = (X_{tr})^{-1} (y_{tr} - \mu(y_{tr}))$$

Linear Regression

$W = (X_{tr})^{-1} (y_{tr} - \mu(y_{tr}))$

X_{tr} is not an invertible matrix but we can compute its pseudoinverse, which is the closest approximation to an inverse.

The pseudoinverse M^* of matrix M is given by:

$M^* = (M^T M)^{-1} M^T$

where M^T is the transpose of M

thus

$W = (X_{tr}{}^T X_{tr})^{-1} (X_{tr})^T (y_{tr} - \mu(y_{tr}))$

and the prediction for the test set is then:

$p = X_{ts} W + \mu(y_{tr})$

Linear Regression for Classification

The One-hot representation

In order to use a regressor for classification (any regressor, not just linear regression) we use a **one-hot** representation of the output.

Suppose y is a 1D array of length n of integers containing the classes in a dataset

As usual, we assume classes are integers in the 0,...,c-1 range

Then the one-hot representation of y is an n-by-c array, where row i consists of 0 everywhere except at position y[i], where its value is one.

For example if y = [4,2,3,1,0,4] and classes are 0,..,4, onehot(y) is given by:

[[0., 0., 0., 0., 1.],
 [0., 0., 1., 0., 0.]
 [0., 0., 0., 1., 0.],
 [0., 1., 0., 0., 0.],
 [1., 0., 0., 0., 0.],
 [0., 0., 0., 0., 1.]])

Linear Regression for Classification

Thus we train the model to predict not y, but the one-hot representation of y: `model.fit(X_train,onehot(y_train))`

When we predict, the model will return the predicted one-hot representation of y_test.

```
p = model.predict(X_test)
```

p will be a 2D array of size `X_train.shape[0]` by `onehot(y_train).shape[1]`

How do we convert to a single prediction?

We could find the Euclidean distance from each row in `p[i]` in `p` to the one-hot representation of each of the classes and assign example `i` to the class with the most similar representation.

This is equivalent to assigning example `i` to the class corresponding to the position of the largest item in `p[i]` (that is, `pred[i] = argmax(p[i])`.

Linear Regression for Classification

For example, the following array would result in predictions [4,2,3,1,0,4]:

[[0.79115577, 0.63147976, 0.39390119, 0.54309383, **1.07130847**],
[0.62835492, 0.02026977, **1.81213046**, 0.0330209 , 0.38278168],
[0.53767076, 0.77982095, 0.91745407, **1.84388367**, 0.04652723],
[0.85631156, **1.76228783**, 0.91840274, 0.15162574, 0.0680549],
[**1.29367029**, 0.01124145, 0.63691936, 0.01732445, 0.05142839],
[0.16763345, 0.83554417, 0.7718447 , 0.17421716, **1.51564816**]])

Beyond Linear Features with Linear Regression

Is it possible to learn a quadratic function such as

$y = 3x^2 - 5x + 10$

using linear regression?

Beyond Linear Features with Linear Regression

What if *y* depends on the square of one or some of the attributes in *X*?

Can we learn a function such as:

p = w[0]*x[0] + ... + w[4]*x[4] + w[5]*x[0]**2 + ... + w[9]*x[4]**2 +w[10]

Beyond Linear Features with Linear Regression

What if *y* depends on the product(s) of two of the attributes in *X*?

Can we learn a function such as:

p = w[0]*x[0]*x[0] + w[1]*x[0]*x[1] + w[2]*x[0]*x[2] + ...+

w[14]*x[4]*x[4] + w[20]

Beyond Linear Features with Linear Regression

What if *y* depends on the product(s) of two of the attributes in *X*?

The linear regression algorithm can only learn linear combinations of the attributes

However, we can create new attributes to augment the original dataset with non-linear combinations of the original attributes